Building a Data-Oriented Operating System

A discovery-based study guide — from bare metal to a windowing system

July 2026

Contents

Executive Summary

This is a **map and a resource atlas, not a recipe.** It names the subjects, the topics inside them, and the design questions you will have to answer — but it deliberately withholds the implementation, because the implementation is the thing you want to derive from theory yourself. Where a reference kernel exists, it is listed as something to *compare against after you have attempted the design*, not to copy from.

The guide is built around four commitments that come from your own framing:

- **Theory into practice.** You already hold the theory — virtual memory, scheduling, synchronization, interrupts, signals, deadlock. The point of this project is to force that theory through the narrow aperture of real silicon and a real toolchain, which is where it finally becomes yours.
- **Data-oriented design, strictly, from the first line.** DOD is treated as a first-class constraint on every subsystem, not a late optimization pass. Retrofitting DOD is painful; the guide front-loads the pattern vocabulary so you design data-first everywhere.
- **Parallel-programming readiness as a through-line.** Large parts of an OS *are* parallel programming at the metal — synchronization primitives, SMP bring-up, memory models, lock-free structures, SIMD state. These are flagged throughout and gathered into a dedicated phase, so the project doubles as preparation for next year's course.
- **Two decisions gate everything.** Your implementation **language** and your target **instruction-set architecture** shape every phase downstream. Phase 0 gives you the criteria; the accompanying discussion gives the opinionated analysis.

The shape is nine phases, from a freestanding binary that can print a character, through memory, scheduling, IPC, userspace, storage, and parallel/GPU concerns, ending at an optional windowing system — plus three cross-cutting threads (DOD discipline, bare-metal testing/debugging, and the parallel-programming spine). **There is no timeline by design.** The phases are dependency-ordered; the pace is yours, and the project is meant to be returned to over a long horizon.

How to Use This Guide

Each phase follows the same rhythm: a short orientation, the **topics to master**, the **design questions you must answer yourself**, and pointers into the resource atlas at the end. Coloured callouts flag the four recurring concerns — key ideas, decision points, parallel-programming payoffs, and data-oriented hooks — plus the occasional reality check where a goal needs honest scoping.

A few working practices that make a long, intermittent project of this size survivable:

- **Design the data before the code, every time.** For each subsystem, write down — in your journal, before touching an editor — what the tables are, what each row holds, how rows are keyed, which fields are hot versus cold, and what the dominant access pattern is. This single ritual is what makes the DOD commitment procedural rather than aspirational.
- **Keep an engineering journal and write ADRs.** Every non-obvious decision (allocator design, scheduler policy, syscall ABI, driver model) gets a short architecture-decision record: the alternatives, the facts that killed the losers, the choice. Future-you, returning after a month away, will need them.
- **Grill each decision before you commit it.** Enumerate the real alternatives and eliminate them on the facts about your data and hardware, not on taste. This is the same single-question interrogation you already apply to plans; apply it to designs.
- **Stand up measurement early.** Before you optimize anything, you need to *see* it: cycle counters, cache-miss and TLB-miss performance counters, and a micro-benchmark harness that runs in your emulator and, ideally, on real hardware. A DOD claim that isn't measured is a guess.
- **Attempt before you read the answer.** Reference kernels (xv6, Redox, the tutorial repos) are powerful, and powerfully tempting. Read them to get unstuck or to compare *after* you've designed a subsystem — not as your first move, or you'll inherit their decisions instead of understanding the space.

> You are not trying to reproduce a specific published kernel. You are trying to become someone who *could have written one* — who can open the architecture manual, extract what the hardware demands, choose a data layout, and defend it. The tutorials get you a working foundation fast; the research skill is what carries you the rest of the way.

Phase 0 — Foundational Decisions & Environment

Nothing here is code. It is the set of choices and the tooling that determine what every later phase feels like. Resolve it deliberately.

Language selection

Your language decides how much control you have over memory layout, whether a runtime and garbage collector sit between you and the hardware, how SIMD is expressed, and how much the compiler helps you stay correct. Because you are committing to strict DOD and heavy optimization, layout control and the absence of a managed runtime dominate the decision. Evaluate candidates against: memory-layout control (struct packing, alignment, struct-of-arrays support), freedom from a mandatory GC/runtime, first-class SIMD intrinsics, cross-compilation ergonomics (this matters enormously for targeting a different ISA), compile-time safety, the maturity of the bare-metal ecosystem, and how naturally the language expresses data-oriented patterns.

The accompanying write-up argues the case concretely — why a garbage-collected language fights you in a kernel (with measured costs from a real research OS), and why the DOD emphasis specifically pulls between two strong systems languages. Make the call there; this guide stays language-neutral so it survives whichever you pick.

Instruction-set architecture: x86-64 vs AArch64

This is the other decision that colours everything. The two architectures diverge in ways that touch nearly every phase:

- **Boot and initial CPU state.** x86-64 carries decades of legacy — the machine wakes in a primitive mode and something must walk it into 64-bit long mode, and you inherit segmentation and the GDT whether you want them or not. AArch64 generally hands you a saner initial state, but you drop through *exception levels* (EL2/EL1) and the platform describes its hardware to you through a **device tree** rather than the largely self-describing PC platform.
- **Privilege model.** Rings 0–3 versus exception levels EL0–EL3. The concepts map; the mechanisms and the register interfaces do not.
- **Interrupt controller.** The x86 IDT plus PIC/APIC versus AArch64 vector tables plus the GIC.
- **MMU and page-table format.** Both are multi-level radix page tables, but the entry formats, control registers (`CR3`/`CR4` vs `TTBR0`/`TTBR1`/`TCR`/`MAIR`), and the clean user/kernel table split on AArch64 differ.
- **Memory model.** x86 is strongly ordered (TSO); AArch64 is weakly ordered. This is not a footnote — it changes how you must write concurrent code, and it is one of the most valuable lessons the project can teach.
- **Hardware discovery.** PCI/PCIe enumeration on the PC platform versus device-tree parsing on typical ARM boards.
- **Your existing leverage.** You already read x86-64 AT&T assembly, which lowers the cost of the x86 path. AArch64 is a new (and, many find, cleaner) ISA to learn.
- **Your actual hardware.** Apple Silicon is AArch64, and its native GPU path is Metal, not CUDA — both facts are genuine inputs here.

Two coherent routes: (1) start on x86-64 to exploit the far denser beginner tutorial ecosystem and your existing assembly knowledge; or (2) commit to AArch64 from the outset using the ARM-native tutorials, closer to the metal you actually own. A third, underrated option: build the foundation on one architecture, then **port it to the other later** — porting is one of the fastest ways to learn what is genuinely architecture-specific versus what was accidental coupling, and it forces the clean arch-abstraction boundary that a serious kernel needs anyway. The companion note takes a position; the topic breakdown below flags where the two diverge so the guide serves either choice.

Toolchain and environment

- Cross-compilation and **custom target specification**: telling the compiler to emit freestanding code for your ISA with no host OS assumptions.
- **Linker scripts**: controlling where sections land in the address space, which matters the moment you have a higher-half kernel.
- An **emulator** as your primary development machine (QEMU's `q35` for x86, `virt` or a specific

board for ARM), and the discipline of also running on real hardware periodically so you don't over-fit the emulator's forgiving behavior.
- A **debugging lifeline**: the emulator's GDB stub, plus serial/semihosting output — you will spend real time diagnosing failures whose only symptom is a silent reset.
- Binary archaeology tools (`objdump`, `readelf`, `nm`, disassembly) and a reproducible image-build pipeline.

Master, before moving on: exactly how a compiled artifact becomes a bootable image, and precisely what state the machine is in at the instant your first instruction runs.

Data-oriented grounding — read before you write code

Internalize the pattern vocabulary now, so that in every later phase you reach for a named pattern instead of reasoning from first principles under time pressure. The core moves you want at your fingertips: **structure-of-arrays** versus array-of-structures; **handles/indices instead of pointers**; **hot/cold data splitting**; **existence-based processing** (membership in a set rather than a per-object boolean flag); **out-of-band booleans**; **encodings over polymorphism** (a tag plus a switch instead of a vtable); and the **relational, normalized model** of state — thinking of your kernel's data as tables you can join, index, and query.

> A kernel is *already* a pile of tables — the process table, the page tables, the file-descriptor table, the inode cache. A file descriptor number is already a handle. This means DOD is the **native idiom** of kernel data, not an imposition from game development. The relational-database framing in Fabian's book maps almost directly onto kernel subsystems, and it will feel less like a technique you're applying and more like a lens that was always appropriate.

The bare-metal language dialect

Whichever language you chose, there is a freestanding dialect to absorb before real work begins: building without the standard library and its hosted assumptions, defining your own entry point, handling panics without an OS beneath you, disabling stack unwinding, expressing volatile and memory-mapped-I/O access correctly, dropping to inline assembly where the ISA demands it, and (later) hooking the language's global-allocator interface to your own heap.

Phase 1 — Bare-Metal Bring-Up

The goal of this phase is modest and non-negotiable: reach a known-good state and get a single character out of the machine. Everything else depends on having a lifeline.

Topics to master

- A **freestanding binary** that links nothing hosted and runs with no OS underneath.
- The **boot path** for your ISA. On x86-64: how a bootloader (a bootloader crate, Limine, or multiboot2) hands control to your kernel already in long mode, and the GDT you inherit. On AArch64: the reset state, dropping from EL2 to EL1, and reading the device tree the firmware hands you.
- Your **first serial output** — a UART driver (16550-family on the PC platform, PL011 on typical ARM) — and a minimal formatted-print facility built on top of it. This is your single most important early tool.
- A **bare-metal test harness**: running assertions inside the emulator and reporting results back

out.

Design questions to answer yourself

- Where does execution begin, and in exactly what CPU state?
- How do you emit a byte before you have "anything" — before memory management, before interrupts?
- What is the smallest abstraction that lets you `print!`-style format text, and what data does it actually own?

> Your serial console is, underneath, a byte ring buffer feeding a device register. Design that buffer's shape and ownership deliberately now — it sets the tone that *every* subsystem starts from its data.

Where to look: the phil-opp series (x86), the Andre Richter AArch64 tutorials, the OSDev wiki's bare-bones and UART pages, and your bootloader's documentation.

Phase 2 — CPU State, Exceptions & Interrupts

Now the machine learns to react to the world and to its own errors. This is also where you meet concurrency for the first time, in miniature.

Topics to master

- The **exception/interrupt model** for your ISA: the interrupt descriptor table and handler-calling convention on x86; the exception vector table and the GIC on ARM.
- The **exception taxonomy** — faults, traps, and aborts — and which are recoverable and how.
- The **double-fault / triple-fault** problem: what happens when a fault handler itself faults, and why you need a *separate, known-good stack* to catch it (the interrupt stack table on x86; banked stack pointers on ARM). A triple fault is a silent reset — designing against it is a rite of passage.
- The interrupt controller: remapping the legacy PIC or driving the local APIC on x86; programming the GIC on ARM.
- The **timer interrupt** (your future scheduler's heartbeat) and **input** (keyboard, or UART-driven).
- The **first concurrency hazard**: a handler and mainline code touching shared state. Interrupt masking, handler reentrancy, and what data an interrupt context may safely touch.

Design questions to answer yourself

- How do you structure the handler table so dispatch is a flat, cache-friendly lookup rather than a chain of conditionals?
- What is the minimal, provably-safe way for a handler to communicate with the rest of the kernel?

> Disabling interrupts is the first exclusion mechanism you'll wield — and understanding *why it is not sufficient on a multi-core machine* is a foundational parallel-programming insight you'll cash in during Phase 4. Note the question now; you'll answer it when a second core exists.

> The IDT / vector table is a flat array indexed by vector number — the purest possible data-oriented structure. Treat it as the template for how kernel dispatch tables should look.

Where to look: Intel SDM Volume 3 (x86 system programming) or the Arm Architecture Reference Manual (exceptions, GIC), and the OSDev wiki's interrupt pages.

Phase 3 — Physical & Virtual Memory

The largest and most rewarding of the early phases. You already know the theory cold; here you implement the mechanism against the actual MMU.

Physical memory

- Parsing the firmware **memory map** to learn what RAM exists and what is reserved.
- **Frame-allocator design** — bitmap versus free-list versus buddy allocator. This is a pure data-structure decision with real consequences for fragmentation and allocation cost.

Virtual memory and paging

- The **multi-level page-table format** for your ISA: 4- and 5-level tables on x86-64; AArch64 translation tables with the `TTBR0`/`TTBR1` user/kernel split and configurable granule sizes.
- **Mapping and unmapping** pages; permission and no-execute bits; a **higher-half** kernel layout.
- The bootstrap problem of **accessing physical frames** the kernel hasn't mapped yet — recursive mapping versus a physical-memory offset map — and the trade-offs of each.

MMU implementation

- Enabling paging and configuring the control registers (`CR3`/`CR4` versus `TTBR`/`TCR`/`MAIR`), and surviving the transition from identity-mapped to higher-half execution.

The TLB

- Coherence and **invalidation** (`invlpg` versus `TLBI`), address-space tagging (`ASID`/`PCID`) to avoid needless flushes, and — as a preview of the multi-core problem — the **TLB shootdown**: keeping other cores' TLBs consistent when you change a mapping.

Kernel heap

- **Allocator designs** in sequence: bump, then free-list, then fixed-size / slab. Hooking the language's global-allocator interface to your heap.
- The **slab allocator as a data-oriented artifact**: segregating free objects by type into homogeneous pools avoids re-initialization cost and fragmentation. This is exactly the kind of type-homogeneous, cache-conscious pooling DOD advocates — and it is standard kernel practice for precisely that reason.

Memory-model foundations

- Acquire/release semantics, fences, and the strong-vs-weak ordering split between x86 and ARM. You need this vocabulary *before* Phase 4, where it becomes load-bearing.

TLB shootdown is a genuine distributed-consistency problem across cores, and the memory-ordering split is the concept most likely to bite you (and most likely to appear in your course). Confronting both in a system you built makes them permanent.

The allocator's *own* metadata is hot data touched on every allocation. Design it to be compact and cache-friendly, and treat the page tables and frame database as the tables they are.

Where to look: OSTEP's virtual-memory chapters for the theory scaffolding, Intel SDM Vol. 3 / the Arm ARM for paging specifics, the phil-opp paging and heap-allocator posts (x86) as a comparison point, and Fabian's relational framing for modelling the tables.

Phase 4 — Threading, Scheduling & Synchronization

This phase *is* parallel programming at the metal. You will hand-build the primitives that your future course treats as black boxes, and you will meet the hardware memory model face to face.

Topics to master

- **Thread abstraction and context switching**: exactly which register and stack state must be saved and restored (architecture-specific assembly), kernel stacks, and the difference between a cooperative yield and a preemptive, timer-driven switch.
- **Two multitasking models**, ideally both: the cooperative `async/await` path (understanding futures, pinning, wakers, and a hand-written executor) and preemptive kernel threads.
- **The scheduler**: the run queue as a data-oriented structure, and a progression of policies — round-robin, priority, multi-level feedback, and a fair scheduler in the spirit of CFS/EEVDF. Per-CPU run queues once you have multiple cores.
- **Synchronization from scratch**, in increasing sophistication: atomics and memory barriers; spinlocks (test-and-set, then test-and-test-and-set, then ticket locks, then queued MCS/CLH locks); mutexes; **semaphores** (counting and binary); condition variables; reader-writer locks; and, as an advanced target, RCU. Understand priority inversion, lock convoys, and fairness.
- **Deadlock in practice**: you know the Coffman conditions and the prevention/avoidance/detection taxonomy — now instrument for deadlock and provoke it, so the theory acquires teeth.
- **SMP**: bringing up additional cores (APIC/SIPI on x86; PSCI or spin-tables on ARM), per-CPU data, inter-processor interrupts, cross-core TLB shootdown, and cache-line awareness including **false sharing**.

Design questions to answer yourself

- What is the exact machine state that defines a thread, and how small can a context switch be?
- Which lock is right for which contention pattern, and how would you *measure* that rather than guess?
- How do you lay out per-CPU data so that two cores never contend on the same cache line by accident?

Almost everything a parallel-programming course covers — atomics, memory ordering, lock design, lock-free reasoning, scheduling across cores — you will have *implemented* here rather than merely read about. This phase alone justifies framing the project as course preparation.

Store thread/PCB state as struct-of-arrays; keep run queues and wait queues as compact, index-keyed structures. And treat false sharing for what it is: a data-layout bug, diagnosable with the counters you set up in Phase 0.

Where to look: OSTEP's concurrency and scheduling chapters; Paul McKenney's freely-available *Is Parallel Programming Hard…* (a.k.a. *perfbook*) for memory models and RCU; Herlihy & Shavit's *The Art of Multiprocessor Programming*; Preshing's writing on memory ordering; and the architecture manuals for atomic and barrier semantics.

Phase 5 — IPC & Kernel-Level Primitives

The concurrency tools you just built, now exposed as first-class kernel objects that user programs can use to cooperate. These are the primitives you named as goals.

Topics to master

- **Shared memory**: mapping the same physical frames into multiple address spaces, and the coherence and synchronization obligations that follow.
- **Pipes**: anonymous/unnamed pipes as ring buffers between related processes, and named pipes (FIFOs) that live in a namespace.
- **Semaphores** as user-facing objects, both named and unnamed.
- **Signals**: delivery, masking, handler invocation, and the async-signal-safety problem.
- **Message passing**: mailboxes/ports, the microkernel-flavored alternative to shared state.
- A **fast-path/slow-path** primitive in the spirit of a futex — uncontended operations resolved in userspace, contention falling into the kernel.

Design questions to answer yourself

- How are these objects named, and how is their lifetime managed?
- How does a blocked waiter get parked and later woken, efficiently, without busy-waiting?
- What does the kernel object table look like, and how do handles index it?

This phase leans on address spaces and processes from Phase 6 — the two interleave in practice. Expect to build the mechanism here and the user-facing surface once processes exist. Sequence them to taste; they are mutually entangled.

Shared memory plus semaphores is the classical substrate of parallel and concurrent computing. Building it is a direct rehearsal of the coordination problems the course will formalize.

Where to look: OSTEP for concepts; Stevens' *Advanced Programming in the UNIX Environment* as a

semantics reference for what these primitives are supposed to *mean*; the OSDev wiki for mechanism.

Phase 6 — Userspace & Processes

The kernel gains a boundary. Everything above this line runs unprivileged, and crossing the line safely is the whole game.

Topics to master

- The **user/kernel privilege boundary**: ring 3 or EL0, how you drop privilege, and the trap-and-return path in both directions.
- The **system-call mechanism** and ABI: `syscall/sysret` (and the legacy `int 0x80`) on x86; `svc/eret` on ARM. How arguments are passed, and how errors return.
- **Per-process address spaces** and the **process-creation model**: fork/exec versus a spawn-style call, and the copy-on-write question.
- **Loading user programs**: parsing and relocating an executable format (ELF), and a minimal user runtime/libc.
- **Safe user↔kernel data transfer**: validating user pointers so a malicious or buggy program can't trick the kernel into touching memory it shouldn't.

Design questions to answer yourself

- What is your syscall ABI, precisely, and how do you keep it fast?
- What is your process model, and how do you isolate and switch between address spaces?
- How do you copy data across the trust boundary without ever trusting a user-supplied pointer?

Process, thread, and file-descriptor tables are the canonical case for struct-of-arrays with **generational handles** — a file descriptor number *is* a handle, and generational indices solve the use-after-free/stale-handle problem cleanly. Fabian's book and floooh's "handles are the better pointers" are the direct references.

Once user processes exist, you schedule them across your cores — the scheduling and per-CPU work from Phase 4 now operates on real, isolated, competing workloads.

Where to look: OSTEP (processes and the process API); the xv6 book as a canonical *minimal* design to study **after** you've attempted your own; the System V ABI and ELF specifications; the OSDev wiki.

Phase 7 — I/O, Drivers, Storage & Filesystems

Connecting to storage — one of your explicit goals. This is where the kernel talks to real (or virtual) hardware and gives programs persistent state.

Topics to master

- A **driver model**: how the kernel abstracts devices; memory-mapped I/O versus port I/O; interrupt-driven versus polled operation; and **DMA**.

- **Bus and device discovery**: PCI/PCIe enumeration and MSI/MSI-X on the PC platform, versus device-tree parsing on ARM.
- A **block-storage driver**: virtio-blk is the sane first target under emulation; AHCI/SATA or NVMe are the real-hardware versions and considerably harder.
- A **filesystem**: a virtual-filesystem abstraction layer, then a concrete filesystem — FAT for interoperability, ext2 for realism, or a custom log-structured design. Directory structures, inodes, and **crash consistency** (journaling / fsck).
- The **buffer/page cache** as a data structure, and asynchronous I/O paths.

Design questions to answer yourself

- What is your device abstraction, and how do drivers register and dispatch?
- Synchronous or asynchronous I/O — and what does that choice do to your kernel's structure?
- What is your caching and write-back policy, and how do you keep on-disk state consistent across crashes?

The block layer between drivers and the rest of the kernel is a bounded-buffer, producer-consumer system — an ideal place to apply the lock-free and bounded-queue techniques from Phase 4/8 to real throughput.

Model the block-request queue, the buffer cache, and the inode/dentry caches as compact, index-keyed structures. This is where data-oriented layout translates most directly into I/O throughput.

Where to look: the OSDev wiki (PCI, AHCI, NVMe, virtio), the VIRTIO specification, OSTEP's I/O and filesystem chapters (including crash consistency), and the xv6 filesystem as a compact reference design.

Phase 8 — Parallelism, SIMD & the GPU Question

Your special-interest phase, and the strongest bridge to next year's course. Here the OS becomes an explicit sandbox for parallel and heterogeneous computing.

SIMD, in the kernel and in userspace

- Vector ISAs: AVX / AVX-512 versus NEON / SVE. Writing vectorized routines — memory copy, checksums, framebuffer blitting — and measuring the speedup honestly.
- The **vector/FPU state-save problem** on context switch: lazy versus eager saving, and the size and cost of modern vector state (`XSAVE` on x86; SVE state on ARM). This is a real, OS-level SIMD concern that pure userspace SIMD practice never exposes you to.

Memory models, hands-on

- Write concurrent code that *visibly reorders*, then fix it with barriers, and contrast x86-TSO against ARM's weak ordering on the very machine you built. Few exercises make the memory model as memorable.

Lock-free and wait-free structures

- Single- and multi-producer/consumer queues, hazard pointers, epoch-based reclamation, and the ABA problem.

Heterogeneous computing and the GPU — scoped honestly

- **Achievable and worthwhile:** framebuffer graphics via virtio-gpu, and — more importantly — understanding the *shape* of GPU interaction: command submission, MMIO doorbells, and DMA ring buffers. You can also define and drive a **simulated compute device** to exercise that shape end-to-end.
- **Research-scale, not a weekend:** a real driver for actual GPU silicon (PCIe BAR mapping, firmware, command rings, on-device memory management) is Linux-DRM-scale engineering. Scope a toy; don't mistake it for a product.

CUDA targets NVIDIA hardware through NVIDIA's proprietary driver, running on a host OS. You will not reimplement that inside your kernel, and it isn't the point. Practice CUDA and the SIMT model as its **own track** on real NVIDIA hardware or cloud notebooks, and let the OS give you the *systems* intuition — parallel scheduling, memory coherence, DMA, asynchronous submission — that makes you a markedly better GPU programmer. On Apple Silicon your native path is Metal rather than CUDA, which is one more genuine input to the architecture decision in Phase 0.

SIMD, memory models, lock-free reasoning, and the systems view of heterogeneous parallelism — implemented and measured in your own OS — is about the strongest possible preparation for a parallel-programming course.

Struct-of-arrays layout is what lets the vector unit stride cleanly through homogeneous data. Batching and treating the render/compute loop as a pipeline of data transforms is the same discipline that pays off in graphics below.

Where to look: the architecture manuals for vector ISAs and vector-state save; McKenney and Preshing for lock-free technique; the virtio-gpu specification; the CUDA C++ Programming Guide and Apple's Metal documentation for the separate GPU track; and Agner Fog's optimization manuals.

Phase 9 — Graphics & GUI

The capstone, and by your own framing the final piece. Large, optional, and best approached only once the system beneath it is solid.

Topics to master

- The rendering ladder: framebuffer access, then 2-D rasterization and compositing, then a display-server / windowing model, then input-event routing, then a widget and layout system — optionally with GPU-accelerated compositing at the top.
- **Data-oriented graphics**: retained versus immediate mode; an entity-component-system for scene and widget state; quad/sprite **batching**; the frame as a pipeline of data transforms; and

damage / dirty-rectangle tracking so you redraw only what changed.

Design questions to answer yourself

- What is your rendering model, and how does state flow through a frame?
- How are input events routed to the right window and widget?
- How do widgets store their state so that layout and drawing stay cache-friendly at scale?

> This is the home turf of the DOD literature. An ECS for widget/scene state, batched draw submission, and dirty-rect tracking are all data-oriented patterns with decades of refinement behind them — exactly the "known patterns" you wanted to offload to rather than reinvent.

Where to look: Fabian's book and the ECS literature (the flecs author's writing is a good, practical entry point); your Metal knowledge for GPU-accelerated compositing; and the OSDev wiki's GUI material.

Cross-Cutting Threads

Three concerns that don't belong to any single phase because they belong to all of them.

Data-oriented design as a through-line

Make the data-first ritual habitual, and keep a running map from the pattern catalogue to your kernel's subsystems: each table becomes struct-of-arrays plus handles; each per-object boolean becomes set membership; each polymorphic hierarchy becomes a tag plus a switch. Measure the effect with the counters from Phase 0 — and know when *not* to bother: genuinely cold, rarely-touched paths don't earn the complexity, and DOD is a tool for the hot path, not a religion.

> Two of your commitments reinforce each other. Kernels are natively relational — tables you join, index, and query — so DOD fits the domain rather than fighting it. And if you chose a language whose type system pushes back on pointer-graph designs, that pressure pushes you *toward* handle/index layouts anyway. The safety discipline and the performance discipline end up pulling in the same direction.

Testing and debugging on bare metal

A distinct skill from either OS theory or your language. Build it in parallel with everything else: unit and integration tests that run inside the emulator; fluency with the GDB stub; serial tracing as your default instrument; disassembly and object-file inspection; emulator tracing; and liberal assertions and invariants. Reproducing a race is its own art — expect to invest in it deliberately.

The parallel-programming spine

Read as a single arc across the guide: the primitives (Phase 4), exposed as IPC (Phase 5), scheduled across cores (Phases 4 and 6), meeting the memory model and lock-free structures and SIMD (Phase 8), and finally the systems intuition for heterogeneous parallelism (Phase 8). That arc maps, topic for topic, onto a serious parallel-programming syllabus — which is the sense in which this project is course preparation rather than a detour from it.

Capability Checkpoints

A **dependency-ordered sequence, not a schedule** — markers to track progress against, deliberately free of any timeline. Your OS can:

1. Print to a serial console.
2. Catch a CPU exception and resume — and survive a double fault without resetting.
3. Take timer interrupts and read input.
4. Enumerate physical memory and hand out frames.
5. Map and unmap its own virtual memory; run higher-half.
6. Allocate on a real kernel heap.
7. Run two kernel threads with context switching.
8. Bring up a second core and run truly in parallel.
9. Block and wake a thread on a semaphore.
10. Pass data between contexts through shared memory and pipes.
11. Drop to userspace and service a system call.
12. Load and run a user program from an executable.
13. Read and write a file on a real (or virtual) disk.
14. Vectorize a hot routine and measure the speedup.
15. Draw to a framebuffer.
16. Composite and route input in a windowed interface.

Resource Atlas

Organized by role. Primary references (the architecture manuals) are where the authoritative answers live; the tutorials get you moving; the reference kernels are for comparison after you've attempted a design.

Primary hardware references

- **Intel 64 and IA-32 Architectures Software Developer's Manual (SDM)** — the authoritative x86-64 reference; Volume 3 is system programming. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- **AMD64 Architecture Programmer's Manual (APM)** — the AMD counterpart; Volume 2 covers system programming.
- **Arm Architecture Reference Manual for A-profile (the "Arm ARM")** — the authoritative AArch64 reference. https://developer.arm.com/documentation/ddi0487/latest
- Board/SoC-specific manuals when targeting real ARM hardware (e.g. the BCM2711 peripherals document for Raspberry Pi 4).
- **OSDev Wiki** — the collective folklore of hobby OS development; indispensable for the gaps the manuals leave. https://wiki.osdev.org

Operating-systems theory

- **Operating Systems: Three Easy Pieces (OSTEP)** — free, and the best modern OS textbook for exactly the concepts you're implementing. https://pages.cs.wisc.edu/~remzi/OSTEP/
- **xv6** (MIT 6.1810) and its accompanying book — a compact, readable teaching kernel to study *after* attempting your own designs. https://pdos.csail.mit.edu/6.1810/
- **Computer Systems: A Programmer's Perspective (CSAPP)** — Bryant & O'Hallaron; systems grounding beneath the OS layer.

Rust bare-metal

- **Writing an OS in Rust** (Philipp Oppermann) — the x86-64 foundation series. https://os.phil-opp.com
- **rust-raspberrypi-OS-tutorials** (Andre Richter) — the AArch64 analog, tutorial-for-tutorial, and excellent. https://github.com/rust-embedded/rust-raspberrypi-OS-tutorials
- **The Embedonomicon** — the freestanding/bare-metal Rust dialect in depth. https://docs.rust-embedded.org/embedonomicon/
- **`core::arch` and portable-SIMD** documentation for kernel/user SIMD. https://doc.rust-lang.org/std/simd/
- **Redox OS** and its book — a full, production-scale Rust OS (microkernel) to study as a mature reference. https://www.redox-os.org · https://doc.redox-os.org/book/

Zig bare-metal (if chosen)

- The **Zig language reference** and standard library — note `MultiArrayList`, a struct-of-arrays container that makes the dominant DOD pattern a built-in. https://ziglang.org/documentation/master/
- Community bare-metal Zig OS projects as references, and Andrew Kelley's DOD talk (below) for the language's data-oriented philosophy in the author's own words.

Data-oriented design

- **Data-Oriented Design** (Richard Fabian) — free online; the canonical text, and the one whose relational/normalized framing maps directly onto kernel tables. https://www.dataorientedesign.com/dodbook/
- **Mike Acton, "Data-Oriented Design and C++"** (CppCon 2014) — the foundational talk on the mindset. https://www.youtube.com/watch?v=rX0ItVEVjHc
- **Andrew Kelley, "A Practical Guide to Applying Data-Oriented Design"** — the most implementation-focused patterns talk: indices instead of pointers, out-of-band booleans, struct-of-arrays to eliminate padding, encodings instead of polymorphism. https://vimeo.com/649009599
- **"Handles are the better pointers"** (floooh) — the handle/index pattern that a memory-managing kernel wants everywhere. https://floooh.github.io/2018/06/17/handles-vs-pointers.html
- **"Operation Costs in CPU Clock Cycles"** (IT Hare) — the latency numbers that make DOD decisions concrete. http://ithare.com/infographics-operation-costs-in-cpu-clock-cycles/
- Entity-component-system writing (the flecs author's essays) for the graphics phase and beyond.

Concurrency & parallelism

- **Is Parallel Programming Hard, And, If So, What Can You Do About It?** (Paul McKenney, "perfbook") — free; memory models, RCU, lock-free technique, the deep end of the field.
- **Preshing on Programming** — the clearest available writing on memory ordering and lock-free correctness. https://preshing.com
- **The Art of Multiprocessor Programming** (Herlihy & Shavit) — the academic reference for concurrent data structures.

IPC & API semantics

- **Advanced Programming in the UNIX Environment** (W. Richard Stevens) — the reference for what pipes, semaphores, shared memory, and signals are supposed to mean.

Storage, filesystems & drivers

- The **VIRTIO specification** (OASIS) for the sane emulated-device path.
- OSDev wiki pages on PCI/PCIe, AHCI, NVMe, and virtio; OSTEP's persistence chapters for filesystem theory and crash consistency.

GPU, SIMD & parallel compute

- **CUDA C++ Programming Guide** (NVIDIA) — the separate GPU/SIMT track. https://docs.nvidia.com/cuda/cuda-c-programming-guide/
- **Metal** documentation (Apple) — your native GPU path on Apple Silicon. https://developer.apple.com/metal/
- **Agner Fog's optimization manuals** — microarchitecture, instruction latencies, and vectorization. https://www.agner.org/optimize/

The language-decision references

- **The benefits and costs of writing a POSIX kernel in a high-level language** (Cutler et al., OSDI '18) — the Biscuit paper; the empirical case on garbage collection in a kernel. https://pdos.csail.mit.edu/papers/biscuit.pdf
- **Redox OS** — evidence that a full OS in a systems language with layout control is not merely possible but mature.